

Solving Non-Linear Equations

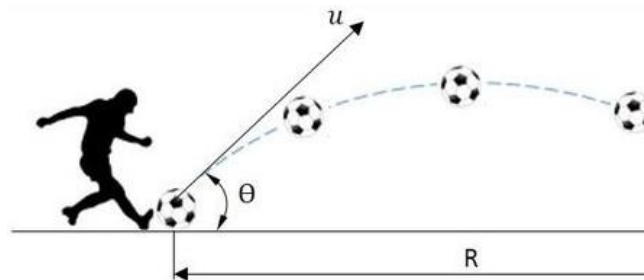
You must complete all the blanks and exercises.

Example Problem

Trajectory Calculation

Problem Statement

A soccer ball is kicked to other player at 20 meters away so he can head the ball. The target height of the ball is 2m. Find the angle at which the player must kick for the pass. Let initial velocity of the ball at the kick is 17m/s, distance $x=20\text{m}$, target height $y=2\text{m}$, and there is no spin.



Formulation of the Solution

The trajectory equation of the ball: Find (θ)

$$y = x \tan(\theta) - \frac{gx^2}{2v^2} \frac{1}{\cos^2(\theta)}$$

with $x=20[\text{m}]$, $y=2[\text{m}]$, $v=17[\text{m/s}]$, $g=9.8[\text{m/s}^2]$

Problem Solving with NM (MATLAB)

[Download matlab code](#)

It is a non-linear equation. First, define a non-linear function in the form of $f(\theta)=0$.

Then, use a non-linear equation solver `fzero()`.

$$f(\theta) = 0 = x \tan(\theta) - \frac{gx^2}{2v^2} \frac{1}{\cos^2(\theta)} - y$$

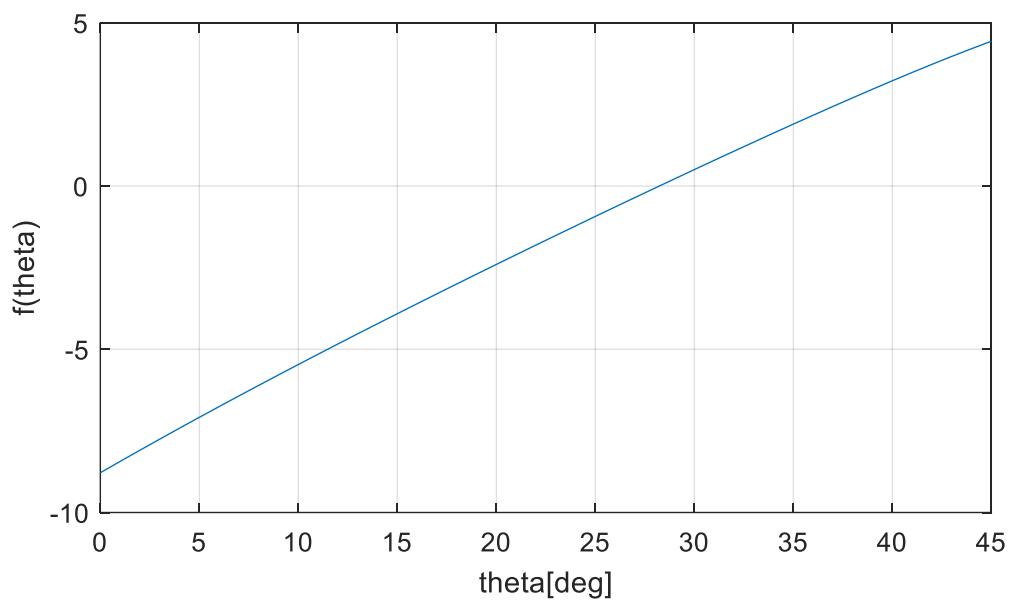
```
t=0:1:45;
plot(t,fnTrajectory(t)); grid on

t0=10;
sol=fzero(@fnTrajectory,t0)

function z=fnTrajectory(t)
    x=20; y=2;v=17; g=9.8;
    z=x.*tand(t)-g.*x.*x./(2.*v.*v.*cosd(t).*cosd(t))-y;
end
```

[Output]:
theta=28.23 [deg]

The estimated solution is in the expected range between 26 and 30 as shown from plotting the graph. Can you estimate the NM solution error in terms of tolerance?



Lecture Objective

We will learn how to make **fzero()** to solve a non-linear equation with one variable, $f(x) = 0$. Then, we will expand the algorithm to solve a system of non-linear equations.

There are several methods explained in the textbook:

- Bisection method: uses only $f(a)f(b) < 0$
- Regular Falsi: uses the values of $f(a)$ or $f(b)$ **and** $f(a)f(b) < 0$
- Newton-Raphson: uses the values of $f(a)$ or $f(b)$ **and** first derivative of $f(x)$
- Secant's method: uses the values of $f(a)$ or $f(b)$ **and** approx.first derivative $f(x)$
- Fixed point iteration: finds the intersection point of two plots $y=g(x)$, $x=g(x)$ iteratively

But, in this lecture, we will concentrate only on

- **Bisection method**
- **Newton-Raphson**
- **Secant method**

Read the textbook for other non-linear methods that are not covered in the lecture.

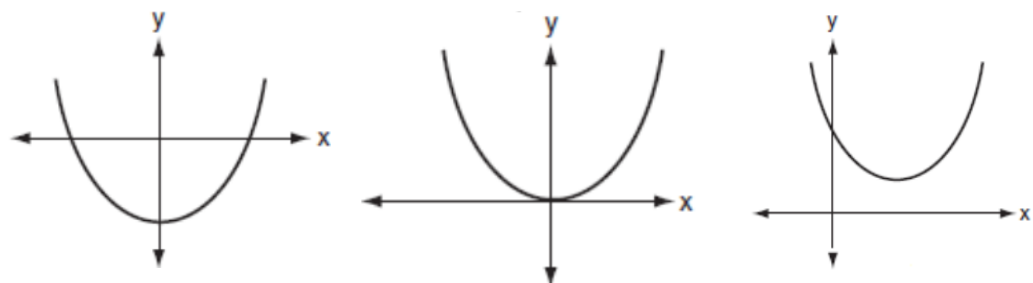
Introduction

The root or the solution of a non-linear equation in the form of $f(x) = 0$ can have

- No root(solution)
- One solution
- Multiple solutions

Examples

- Polynomials
 - Quadratic polynomials: $f(\lambda) = a\lambda^2 + b\lambda + c = 0$. A simple non-linear equation that can be solved analytically. Does the root always exist? If we shift the curve in the y-axis?

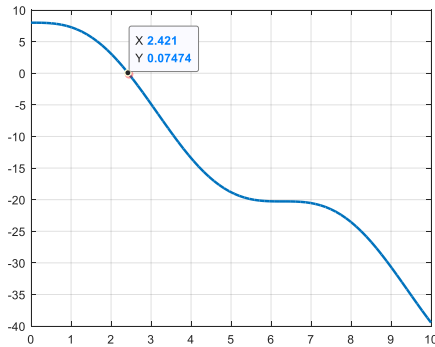


- High-order polynomials: can you solve analytically when the order is a high number?

Some non-linear equations such as a quadratic polynomial can have the analytical solution form. But when the analytical solution does not exist or hard to be obtained, then we should use numerical method to solve.

- Example: $f(x) = a - b(x - \sin x) = 0$

$F(x) = 8 - 4.5(x - \sin(x))$;



```
F = inline('8-4.5*(x-sin(x))');
x=0:0.001:10;
plot(x,F(x)); grid on

sol=fzero(F,2.5)
```

[SOLUTION]
sol = 2.4305

Note that we will apply a NM algorithm to find one solution at a time, even the equation has multiple roots.

Estimation of Errors in NM solution

The non-linear methods are iterative process that can be stopped when the estimated solution is within acceptable error. How can we estimate the solution error when we do not know the true solution?

- True error of x : $True\ error = X_T - X_N$

There are several options to estimate the errors of the numerical solution of a non-linear system of $f(x) = 0$. Use any of them to stop the algorithm iteration with your choice of acceptable tolerance or error bound.

- Tolerance in $f(x)$: $Tol_{f(x)} = |f(x_T) - f(x_N)| = |0 - \varepsilon| = \varepsilon$
- Tolerance in x for bracketing method: $Tol. = \left| \frac{b-a}{2} \right|$, where $x(k)_N = \frac{a+b}{2}$
- Estimated Rel. Error: $\left| \frac{X_T - X_N}{X_T} \right| \approx \left| \frac{X_N^{(k)} - X_N^{(k-1)}}{X_N^{(k-1)}} \right|$, as $X_N \rightarrow X_T$

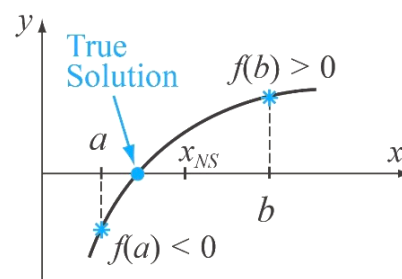
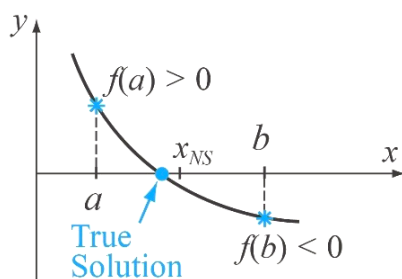
Bisection method

The bisection method is a bracketing method that uses an interval which includes the true solution x_T in $[a, b]$, where a, b are the upper & lower bound for the solution. Here, we have to assume or check that the function $f(x)$ is

- continuous and
- it has a solution within the interval $[a, b]$

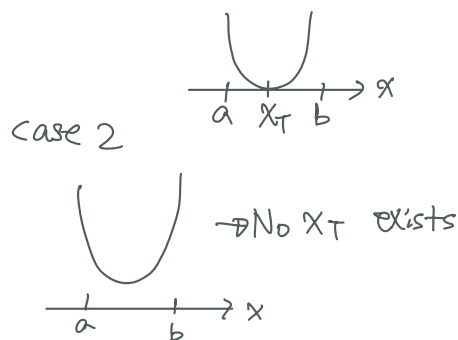
It successively reduces the interval range with iterations based on the following simple concept.

- If $f(x)$ is continuous in $[a, b]$ and the solution x_T is between $[a, b]$, then the sign of $f(a)$ and $f(b)$ are opposite $\rightarrow$ **$f(a)f(b) < 0$** .



- This algorithm fails if $f'(x_T) = 0$.

Exercise: Case 1.



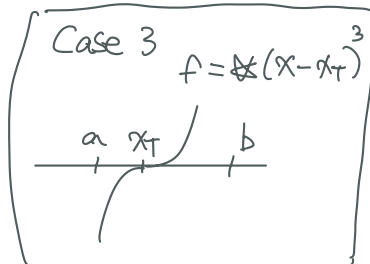
In those cases, no matter how you choose a and b , $f(a) \times f(b)$ will always be bigger than or equal to 0, which means you cannot assume the next x_N .

- Otherwise, this bracketing method always converges with the rate of

$$|x_T - x_n| \leq \left| \frac{b_0 - a_0}{2^{n+1}} \right|$$

- Thus, the convergence rate will be faster if we choose intervals closer to the true solution i.e., (a_0 and b_0 are very close to each other)

But in the following case,

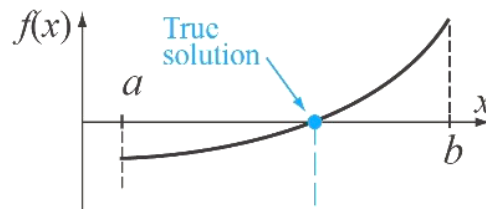


$$f'(x_T) = 0$$

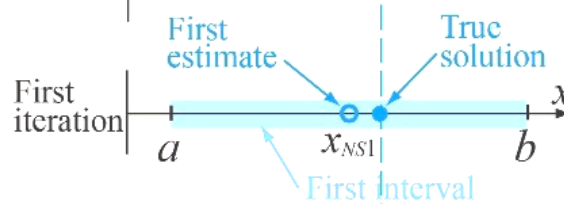
and it will still converge to the true value.

Algorithm:

(0) Initially, observe the zero-crossing $f(x)=0$ and check if it has a root



(1) Then, choose an initial interval $[a, b]$ such that $f(a)f(b) < 0$



(2) Estimate the next solution as $x_N(1) = \frac{a+b}{2}$, // $k = 0$

(3) Check whether x_T is closer to a or b :

Update the next estimation $x_N(k+1)$ with the new range $([a, x_N(k)]$ or $[x_N(k), b])$

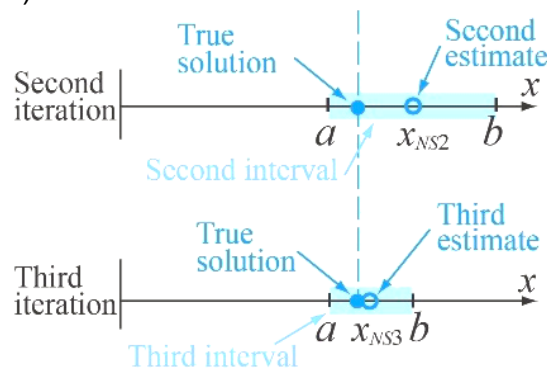
If $f(a)f(x_N) < 0$, then x_T is within $a < x_T < x_N$. $x_N(k+1) = \frac{a + x_N}{2}$

If $f(a)f(x_N) > 0$, then x_T is within $x_N < x_T < b$. $x_N(k+1) = \frac{x_N + b}{2}$

(4) Check the tolerance or rel. error.

$$Tol_{f(x)}(k) = \frac{|f(x_N)|}{|f'(x_N)|}$$

(5) Repeat from step 3) until the tolerance condition is satisfied.



Exercise:

Write a pseudocode for the bisection method

$N_{\max}, \text{tol}, k, \text{ep}$ (initialization) while($k < N_{\max}$ and $\text{ep} > \text{tol}$) { $x_N = \frac{a+b}{2}$ if ($f(a) \times f(x_N) < 0$) $b = x_N$ if ($f(b) \times f(x_N) < 0$) $a = x_N$	$\text{ep} = f(x_N) $ $k++$ end while }
--	---

MATLAB program for the bisection method

See MATLAB demo

Write a C-program for the bisection method

<pre>// double bisection(double a0, double a^{b0}, tol); // it returns the final solution x_N. Input arguments are a0, b0, tol. // Also, need to define a function for the equation $f(x)$</pre>	<i>e.g. double myFunc(double x);</i>
---	--------------------------------------

```
double Func( double x )  
{  
    double F;  
    F = eq w.r.t x;  
    return F;  
}
```

```
double bisection( double a0, double b0, double tol )  
{  
    int Nmax = 200;  
    int k = 0;  
    double ep = 1234.0;  
    double xN;
```

```
    while( k < Nmax && ep > tol ) {  
        xN = (a+b)/2;  
        if( Func(a) * Func(xN) < 0 )  
            b = xN;  
        else  
            a = xN;  
        ep = fabs( Func(xN) );  
        if( ep == 0 )  
            break;  
        k++;  
    }  
    return xN;  
}
```

Newton-Raphson method

This method also solves for the root of an equation of the form $f(x)=0$. In contrast to bisection method, this uses the slope of the equation to converge to the solution at a much faster rate.

This method requires the $f(x)$ is

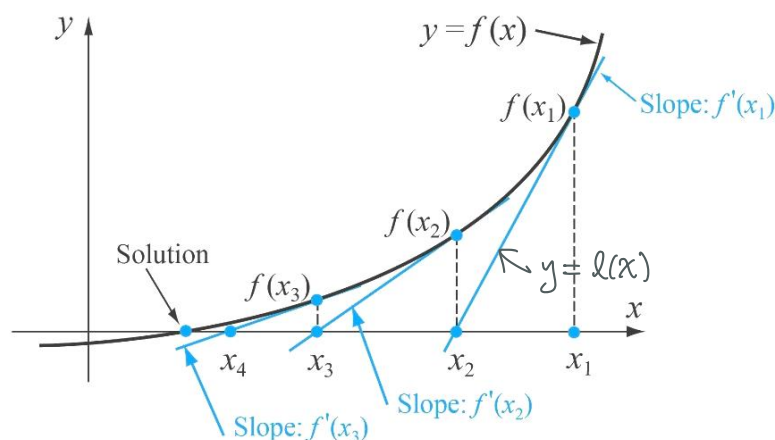
- continuous and
- differentiable
- start at the initial point near the true solution

It is an 'open method', and it is generally much faster in convergence than Bisection or Regular Falsi for it is using the equation slope.

- Convergence rate: (bisection vs Newton-Raphson)

$$|x_T - x_k| \leq \left| \frac{b_0 - a_0}{2^{n+1}} \right| \text{ vs } |x_T - x_{k+1}| \leq c(\delta) |x_T - x_k|^2$$

Algorithm



- 1) The solution process starts by choosing the initial point, x_1 , as the first estimate. This should be near the true solution to avoid divergence or converging to another solution.
- 2) The second estimate, x_2 , is obtained by taking the tangent line to $f(x)$ at $(x_1, f(x_1))$. The intersection point of the tangent line to the x -axis becomes the next estimation, x_2 .

$$l(x_{k+1}) = 0$$

$$f'(x_k) = \frac{l(x_{k+1}) - f(x_k)}{x_{k+1} - x_k}$$

$$x_{k+1} - x_k = - \frac{f(x_k)}{f'(x_k)}$$

$$\therefore x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

- 3) Update the estimation. For the general case, the next estimate solution x_{k+1} is the x -axis intersection of tangent line of $f(x_k)$:

$$f'(x_k) = \frac{f(x_{k+1}) - f(x_k)}{(x_{k+1} - x_k)} \quad . \quad // \quad \text{Note } f(x_{k+1}) = 0$$

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Also, it can be expressed as

$$x_{k+1} = x_k + h_k, \quad h_k = - \frac{f(x_k)}{f'(x_k)}$$

- 4) Repeat until the estimation error is smaller than a specified value

$$\left| \frac{x_{k+1} - x_k}{x_k} \right| \leq \varepsilon \quad \text{or} \quad |f(x_k)| \leq \delta$$

Exercise:

Write a pseudocode for the newton's method

```
Nmax, tol, ep, xN    initialization  
while ( k < Nmax and ep > tol )  
    double h = -f(xN)/f'(xN)  
    xN = xN + h  
    ep = |f(xN)|  
    k++  
end while
```

MATLAB program for the newton's method

See MATLAB demo

Write a C-program for the newton's method

```
// float newton()  
// It should return the solution of xN. Input arguments are x0, tol.  
// Assume float func(float x) and float dfunc(float x) are pre-defined.  
float newton(double x0, double tol)  
{  
    int Nmax = 200;  
    int k = 0;  
    double ep = 123.0;  
    double xN = x0;  
    double h;  
    while (k < Nmax && ep > tol)  
    {  
        h = -func(xN)/dfunc(xN);  
        xN = xN + h;  
        ep = fabs(func(xN));  
        if (ep == 0)  
            break;  
        k++;  
    }  
    return xN;  
}
```

More notes on Newton-Raphson method

Interpretation of newton's method using Taylor Series

- Taylor series: $f(x) = f(x_1) + f'(x_1)(x - x_1) + \frac{1}{2!}(x - x_1)^2 f''(x_1) + \dots$
 $f(x_T) = 0$ and x_1 is near x_T
 $f(x_T) = f(x_1) + f'(x_1)(x_T - x_1) + \dots$
 $f(x_T) \approx f(x_1) + f'(x_1)(x_T - x_1) \approx 0$
$$x_T = x_1 - \frac{f(x_1)}{f'(x_1)} \rightarrow \text{approximate solution}$$
- Other interpretation: Let x_1 is near x_T
 $f(x_T) = f(x_1 + h) = 0$
$$= f(x_1) + f'(x_1)h + f''(x_1)\frac{h^2}{2} + \dots$$

$$f(x_1) + hf'(x_1) \approx 0$$

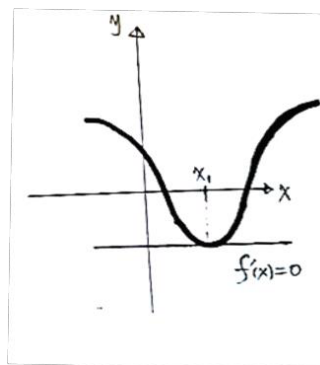
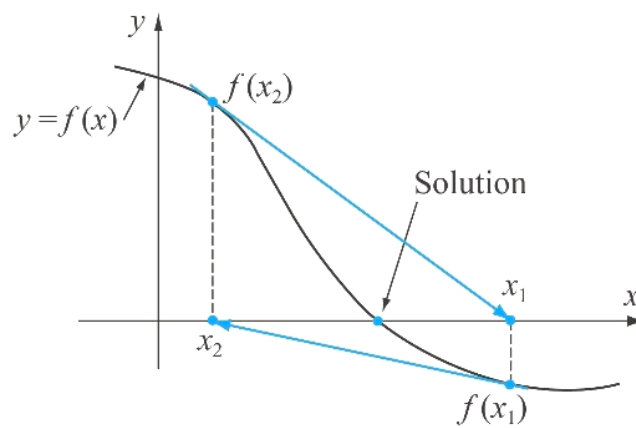
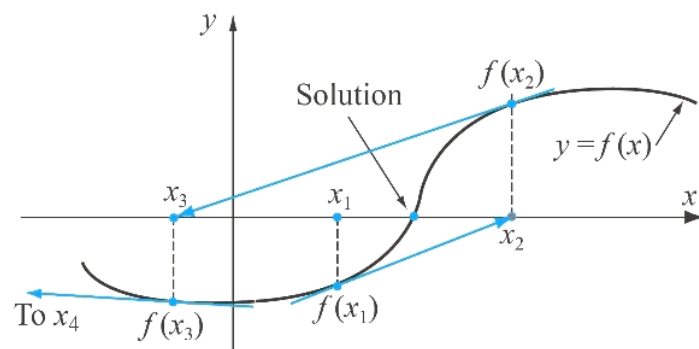
$$h \approx -\frac{f(x_1)}{f'(x_1)}$$

$$x_T \approx x_1 + h = x_1 - \frac{f(x_1)}{f'(x_1)}$$

Convergence of Newton's method

- Does it always converge to the true solution, $\lim_{k \rightarrow \infty} x_k = x_T$?
- Assume $f'(x_T) \neq 0$ and f, f', f'' are continuous nearby x_T
- Then $|x_T - x_{k+1}| \leq c(\delta)|x_T - x_k|^2$
where the initial point x_0 satisfies $|x_T - x_0| \leq \delta$
and $c(\delta) = \frac{1}{2} \frac{\max |f''(x)|}{\min |f'(x)|}, \delta > 0$
- Quadratic convergence
As $\delta \rightarrow 0$, $c(\delta) = \frac{1}{2} \left| \frac{f''(x_T)}{f'(x_T)} \right|$ and $c(\delta)|x_T - x_k|^2 \rightarrow c(\delta)\delta^2 \rightarrow 0$
- But if the initial guess is not close enough to x_T , it may not converge
- Converges if $f'(x), f''(x)$ are continuous, $f'(x_T) \neq 0$ and x_1 is close to x_T
- What if $f'(x)$ is difficult to analyze? Use numerical or secant method

Diverging cases



Secant method

x_k and x_{k-1} 은 주어져야겠지

Similar to Newton's method but it does not directly use the derivative of $f(x)$, $f'(x)$.

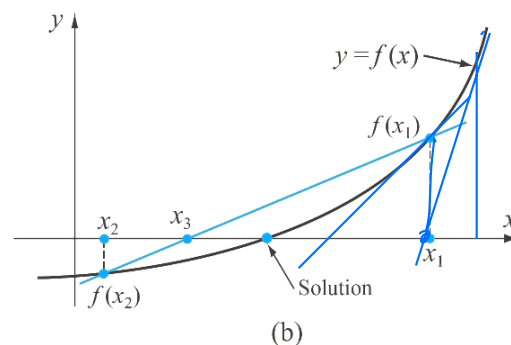
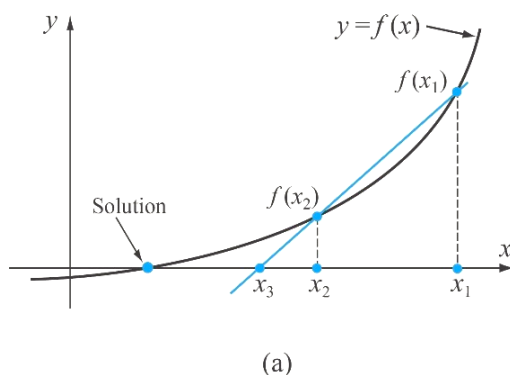
Instead, it estimates the value of $f'(x)$ with two points in the neighborhood of the solution:

Newton's method	Secant method
$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$	$x_{k+1} = x_k - \left(\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \right) f(x_k)$

It is based on a simple numerical method for finding the derivative (two-point backward), which we will learn in later chapter.

This method is useful when the $f'(x)$ is difficult to obtain analytically for applying the Newton's method.

The secant line formed by two points $f(x_1)$ and $f(x_2)$ that passes x -axis, and x_2 can be chosen arbitrarily. The rest process is similar to Newton's method, except the slope is estimated using the secant line.



Q). What happens if the sign of $f(x_1)$ and $f(x_2)$ are different? ~~Does~~ the two points need to be on the opposite side?

No, it can cause divergence rather than converging to x_r

as shown in Fig. (b) above.

Q) Compare the convergence rate with the Newton's method

$$|e_{n+1}| \leq c |e_n|^{1.62} \text{ vs } |e_{n+1}| \leq c |e_n|^2$$

Secant

Newton

Newton's method has a faster convergence rate

Q) Compare the secant method to Newton's method when x_k and x_{k-1} are very close values.

In this case, the Secant method may converge to the true value more quickly than the Newton's method.

Because there is only one initial value for Newton's method

If x_k and x_{k-1} are very close to each other,

that means the slope was extremely sharp.

Algorithm

(0) Initially, observe the zero-crossing $f(x)=0$, check if it has a root, and $f'(x) \neq 0$

(1) Choose two initial values, x_0, x_1 .

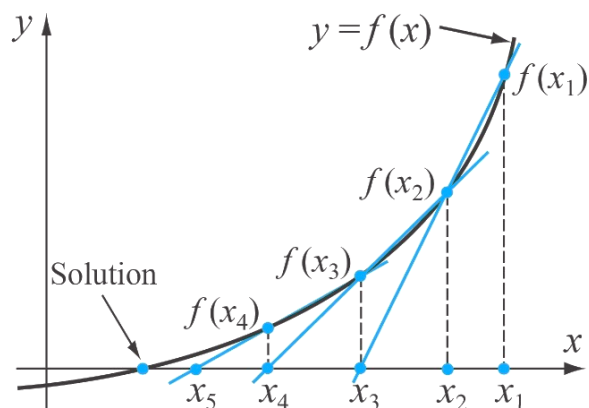
(2) Estimate the next solution x_2

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = x_1 - \left(\frac{x_1 - x_0}{f(x_1) - f(x_0)} \right) \times f(x_1)$$

The estimation of the next solution x_{k+1}

$$x_{k+1} = x_k - \left(\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \right) \times f(x_k)$$

(3) Repeat until it converges to the target tolerance



Note

- The convergence can fail if the secant slope ≈ 0 near the root. It can jump a large amount.

Exercise:

Write a pseudocode for Secant method

```
tol, Nmax, ep,  $x_{N-1}$ ,  $x_N$  initialization  
while (  $k < Nmax$  and  $ep > tol$  )  
    dfunc =  $(x_N - x_{N-1}) / (f(x_N) - f(x_{N-1}))$   
     $x_{N-1} = x_N$   
     $x_N = x_N - dfunc \times f(x_N)$   
     $ep = |f(x_N)|$   
     $k++$ , end while
```

MATLAB program

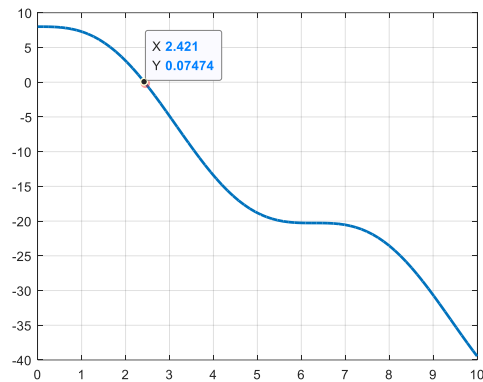
See MATLAB demo

Write a C-program for the Secant method

```
// float secantfzero()  
// input arguments of x0, x1, tol.  
// returns  $x_N$   
// assume float func(float x) is pre-defined.  
float secantfzero(double x0, double x1, double tol)  
{  
    double ep = 100;    double dfunc;  
    int k = 0, Nmax = 200;  
    double  $x_N = x_1$ ,  $x_{N-1} = x_0$ ;  
    while (  $k < Nmax$  &&  $ep > tol$  )  
    {  
        dfunc =  $(x_N - x_{N-1}) / (func(x_N) - func(x_{N-1}))$ ;  
         $x_{N-1} = x_N$ ;  
         $x_N = x_N - dfunc \times func(x_N)$ ;  
         $ep = fabs(func(x_{N-1}))$ ;  
        if (ep == 0)  
        break;  
         $k++$ ;  
    }  
    return  $x_N$ ;  
}
```

Example: Comparison of convergence speed

$$F(x) = 8 - 4.5(x - \sin(x));$$



```
F = inline('8-4.5*(x-  
sin(x))');  
dF=inline('-4.5*(1-cos(x))');
```

```
Nmax = 20; tol = 1e-6;  
x=0:0.001:10;
```

```
plot(x,F(x)); grid on  
sol=fzero(F,2.5)
```

- For Tol=1e-6
- Fzero(MATLAB): sol =2.4305
- Bisection: sol =2.4305, k=16
- NewtonRhason: sol =2.4305, k=2